What Does Reproducible LLM Serving Cost? Routes to Batch Invariance in vLLM on NVIDIA GPUs

Isaac M. Thompson
Brigham Young University

Abstract

A language model served in batches is not deterministic: the same greedy request can produce different tokens depending on which other requests share its batch and whether its prefix is cached. This undermines reproducible evaluation and makes the sampler and the trainer disagree in reinforcement learning. vLLM's batch-invariant mode removes the dependence, but on an H100 it cuts decode throughput at batch 32 by 63.1% for Qwen2.5-7B. We show that this cost is mostly a kernel-tuning gap rather than an inherent price of determinism: for Llama-3.1-8B, whose shapes vLLM has tuned, the same mode costs −0.5%. We compare three routes to closing the gap. Tuning vLLM's own batch-invariant Triton kernel for Qwen2.5-7B keeps every output bitwise identical (0 of 146,010 rows differ across batch sizes; 0 of 30 requests change across serving conditions) and reduces the loss to 10.0%, and in online serving it raises decode-heavy throughput by 101.7% over stock mode; we have contributed these configurations to vLLM. Routing Linear layers to cuBLAS instead reduces the loss to 0.2%, and cuBLAS is batch-invariant for wide projections, but the property is undocumented and fails for narrow layers such as mixture-of-experts routers in ways that depend on the inner dimension, the cuBLAS environment and the batch size, so we advise against relying on it. On Blackwell, cuBLAS's documented experimental batch-invariance mode is bitwise batch-invariant on every shape we tested with PyTorch's default settings, but not in combination with the PyTorch settings that vLLM's batch-invariant mode applies, which break it for odd output widths. Along the way we identify a sliding-window attention bug that makes vLLM's batch-invariant mode non-invariant for Gemma-2 on H100, now fixed upstream.

1 Introduction

Greedy decoding is often assumed to be deterministic, but in a batched inference server it is not. The same request, served alone or alongside unrelated requests, with or without a cached prefix, can produce different tokens, because the floating-point reduction order of matrix multiplication and attention kernels depends on the shape of the batch [He and Thinking Machines Lab, 2025, Yuan et al., 2025]. In our measurements on vLLM [Kwon et al., 2023], 41 of 100 greedy math requests to Qwen2.5-7B-Instruct changed tokens across these conditions. For evaluation this means a system's outputs depend on server load; for reinforcement learning it means the policy that samples a rollout and the policy that is trained on it are numerically different models. In a machine translation pilot (4 systems, WMT23 English–German and Chinese–English, 5 serving configurations that differ only in how requests are batched), 7%–37% of segments changed between configurations. At this scale the conclusions held: none of 24 pairwise system comparisons changed sign or significance, and corpus COMET moved by at most 0.49 points. Outputs, and anything computed on individual outputs, are not reproducible without batch invariance.

vLLM's batch-invariant mode removes this dependence by replacing batch-dependent reductions with fixed-order kernels. Its cost varies widely by model. On an H100 it reduces decode throughput

at batch 32 by 63.1% for Qwen2.5-7B-Instruct and 59.2% for Gemma-2-9B, but by −0.5% for Llama-3.1-8B and 9.1% for Qwen3-8B, whose Linear shapes have hand-tuned kernel configurations in vLLM [vLLM contributors, 2026b]. The difference is the kernel configuration, not determinism.

We ask what it takes to make batch-invariant serving cheap for a model without tuned configurations, and compare three routes on Hopper and Blackwell GPUs. We contribute:

- a measurement showing that most of the cost of vLLM's batch-invariant mode on untuned models comes from its default matrix-multiplication configuration, and that tuning closes most of it without giving up invariance, with tuned configurations for popular models contributed to vLLM (Section 5);
- a bitwise characterization of when cuBLAS is and is not batch-invariant under the mode's settings, across shapes, inner dimensions, environment variables and three GPU types, and an end-to-end evaluation of routing Linear layers to it (Section 6);
- a test of cuBLAS's documented batch-invariance mode on Blackwell (Section 7);
- the diagnosis of a Gemma-2 failure of vLLM's batch-invariant mode, fixed upstream [vLLM contributors, 2026c] (Section 8), and practical recommendations (Section 9).

Every pass/fail rule was written before the corresponding run, and all code, raw outputs and rules are public.

2 Background

A batch-invariant kernel computes each row of its output with the same sequence of floating-point operations regardless of how many other rows are in the batch. Reductions whose split depends on batch size (split-K matrix multiplication, split-KV attention, and some normalization kernels) break this property [He and Thinking Machines Lab, 2025]. vLLM's batch-invariant mode, enabled with `VLLM_BATCH_INVARIANT=1`, pins attention to a single split, replaces reductions with fixed-order kernels, disables reduced-precision reductions, prefers cuBLASLt, and pins collective communication to fixed algorithms. Unquantized Linear layers run a persistent Triton matrix-multiplication kernel on every CUDA GPU, a choice made after gpt-oss failed batch invariance with cuBLAS on H100 [vLLM contributors, 2026a]. The kernel's configuration comes from a table keyed by GPU family and weight shape (N, K): each entry fixes one reduction tile `BLOCK_K` for the shape and chooses the remaining tile sizes per batch-size bucket. Because `BLOCK_K` alone fixes the reduction order, any entry of this form is batch-invariant by construction. Shapes without an entry use a single default configuration for every batch size.

3 Setup and protocol

vLLM main (0.33.0rc1.dev454), PyTorch 2.14 with CUDA 13.0 and cuBLAS 13.1, bf16, NVIDIA H100 80GB unless noted; B200 and H200 for the bitwise tests. *Determinism*: each request is served 8 times in random order under four conditions (cold or warm prefix cache, alone or with 7 unrelated requests, each twice), and every pair of the 8 outputs is compared token by token; top-20 logprobs are also recorded. Prompts are few-shot GSM8K and MATH-500 problems. A control with batch-invariant mode off must diverge for a test to count. *Cost*: decode throughput loss at a fixed batch size against the mode off, measured with two output lengths so that prefill cancels. *Bitwise tests*: a row multiplied alone is compared with the same row inside batches of 1 to 8192 rows. Pass/fail rules were written before each run (repository `PROTOCOL.md`).

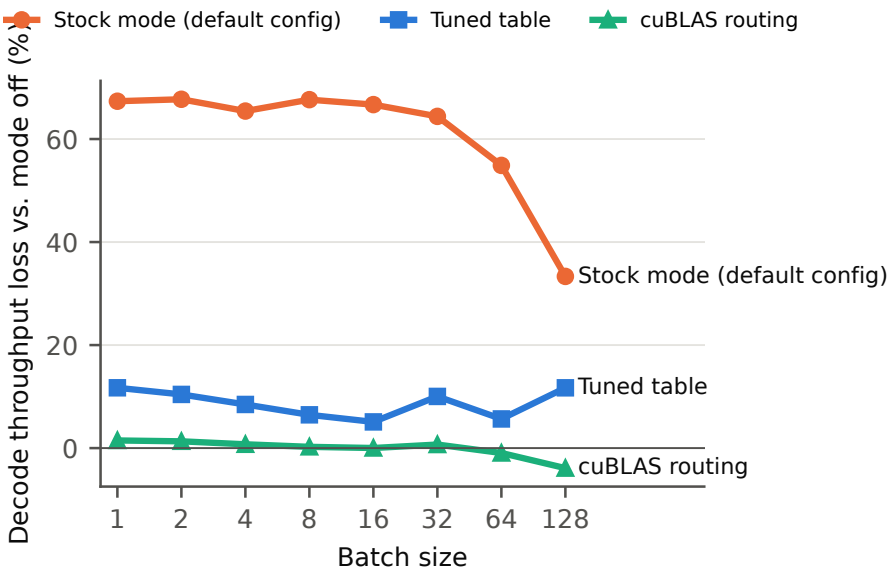


Figure 1: Qwen2.5-7B decode throughput loss against batch-invariant mode off on H100, by batch size. Stock mode uses vLLM's default configuration for every untuned shape; the tuned table adds our per-shape entries and keeps invariance by construction; cuBLAS routing is fastest but its invariance is undocumented (Section 6). Stock and tuned: one job each; cuBLAS routing: the main cost measurement.

4 Where the cost comes from

The default configuration uses 128-row tiles at every batch size. At decode batch sizes most of each tile is padding, so the kernel does several times the necessary work. Tuned entries choose small tiles for small batches. This explains the spread above: the models with tuned entries lose −0.5% and 9.1%, the untuned ones 63.1% and 59.2%.

5 Route 1: tune the Triton kernel

We tuned the five Qwen2.5-7B Linear shapes in vLLM's table format: for each shape one `BLOCK_K` (64 or 128), and per batch-size bucket the fastest tile sizes, warp count and pipeline depth, chosen by timing on an H100. The tuned kernels run 2.10–3.37× faster than the default at batch sizes up to 32, within 0.97–1.15× of cuBLAS there, and 1.01–1.51× slower than cuBLAS on the attention and MLP projections at batch sizes of 256 and more.

Invariance is unchanged: 0 of 146,010 rows differ between a row alone and inside batches of 1 to 8192 rows, over all 5 shapes, and with the table loaded in vLLM 0 of 30 requests change across serving conditions. Decode cost falls from 33.3%–67.7% to 5.1%–11.7% across batch sizes up to 128 (Figure 1, Table 1); at batch 32 it is 10.0% against 64.4% for stock mode in the same job. Our preregistered target was at most 5%, so tuning alone does not close the whole gap; what remains is the kernel's speed at larger batch sizes, which a faster batch-invariant matrix-multiplication kernel would address.

Batch size	1	2	4	8	16	32	64	128
Stock mode (default config)	67.3%	67.7%	65.4%	67.6%	66.7%	64.4%	54.9%	33.3%
Tuned table	11.7%	10.4%	8.5%	6.5%	5.1%	10.0%	5.7%	11.7%

Table 1: Qwen2.5-7B decode throughput loss against batch-invariant mode off on H100, with vLLM's default configuration and with our tuned table. One job per arm.

Serving benchmark and upstream contribution. In an online serving benchmark (`vllm bench serve`, random prompts, concurrency 32, fresh server per run), the tuned table raised decode-heavy output throughput by 101.7% over stock mode, winning 3 of 3 alternating rounds; the loss against the mode off fell from 59.9% to 19.1%. On a prefill-heavy workload the gain was 11.3%. We contributed the configurations to vLLM in the format of earlier tuned-table contributions [vLLM contributors, 2026b,d], with a unit test that checks configuration selection and exact output equality on both sides of every bucket boundary (40 tests pass) [Thompson, 2026b].

Popular models. At a maintainer's request, the upstream contribution covers shapes from widely used models instead: the bf16 layers of GLM-5.3 and DeepSeek-V4.1 that reach this kernel under batch invariance (routers, sparse-attention indexer projections, DeepSeek-V4.1's compressor and `lm_head` per tensor-parallel rank; the rest of both models is FP8 or FP4 and uses other kernels), and gpt-oss's attention, routers and `lm_head`. Over all 15 shapes, 0 of 438,030 rows differ in the bitwise check, and the tuned kernels are 1.08–3.04× faster than the default at every bucket. End to end the gain is small, because most of these models' computation runs in quantized expert kernels that the table does not touch: decode-heavy serving throughput rises by 4.3% for gpt-oss-20b and 1.9% for gpt-oss-120b, faster in 3 and 3 of 3 rounds; for gpt-oss-20b the loss against the mode off falls from 22.8% to 19.5%. Tuning pays off in proportion to the share of a model's computation in unquantized Linear layers: dense bf16 models gain the most.

6 Route 2: route Linear layers to cuBLAS

Is cuBLAS batch-invariant? We multiply each row alone and inside batches of 1 to 8192 rows, at every Linear shape of six models, in bf16 and fp16, and compare rows near the start and end of the batch (Table 2). Under batch-invariant mode's settings, no row differed on any of 7 H100 builds or 2 B200 builds (4 cuBLAS versions; 0 of 769,920 comparisons), including with a second process saturating the GPU. With PyTorch defaults, between 6,784 and 8,544 of 38,496 rows differed on H100, and 19,544 on B200, so the test detects batch dependence when it exists.

Which shapes? A denser test checked every batch size from 1 to 1024 and every sixteenth up to 8192, on 37 shapes including mixture-of-experts routers and a sweep of narrow output widths N. On H100, at $K = 2880$ and 4096, every shape with $N \leq 96$ is batch-variant under the mode's settings, and every shape with $N \geq 128$ is batch-invariant at every batch size. A finer sweep (Figure 2) shows that the boundary also depends on K and on the cuBLAS workspace environment variables, which vLLM sets at runtime. With them, widths 64 and 96 are invariant at $K = 512$, 896, 1024, 1536, 2048 and 8192 but not at 2880, 4096 and 5120, and widths from 97 to 127 are invariant only when they are multiples of 8. Without them, every tested width from 64 to 1024 fails at $K = 2880$, 4096 and 5120, and only widths of 2048 or more pass at every K. An H200 gives the same result as the H100 on 96 of 96 shapes with the variables and 96 of 96 without. The gpt-oss-20b router ($N = 32$, $K = 2880$) diverges from batch size 23 in bf16 and 2 in fp16, and on H200 its failures are

GPU, torch build	BI flags	Flags only	Defaults
B200, torch t2.13.0+cu130	0	0	19,544
B200, torch t2.14.0+cu130	0	0	19,544
H100, torch t2.11.0+cu128	0	0	7,344
H100, torch t2.12.0+cu126	0	0	6,784
H100, torch t2.13.0+cu126	0	0	6,784
H100, torch t2.13.0+cu129	0	0	7,344
H100, torch t2.13.0+cu130	0	0	8,544
H100, torch t2.14.0+cu126	0	0	6,784
H100, torch t2.14.0+cu130	0	0	8,544
H100, torch t2.14.0+cu130 + GPU load	0	0	8,544

Table 2: Rows that differ between a row multiplied alone and inside a batch, out of 38,496 per cell. BI flags: environment variables and PyTorch flags as batch-invariant mode sets them. Flags only: PyTorch flags without the cuBLAS environment variables. Defaults: PyTorch defaults (control).

Configuration	Plain cuBLAS	Mode off	Markers
Qwen2.5-7B, TP=2	0 / 100	38 / 100	2
Qwen2.5-7B, TP=4 (TP=2 control)	0 / 100	38 / 100	4
Qwen2.5-7B, long context, chunked prefill	0 / 50	29 / 50	1
Qwen2.5-7B, seeded sampling	0 / 100	40 / 100	1
Qwen2.5-32B, TP=2	0 / 100	76 / 100	2
Mistral-7B	0 / 100	4 / 100	1
Gemma-2-9B	39 / 100	60 / 100	1

Table 3: Requests whose 8 runs were not all identical, with Linear layers on cuBLAS under batch-invariant mode (plain cuBLAS) and with batch-invariant mode off (control). Markers count processes that loaded the routing change.

interleaved with passing batch sizes [Thompson, 2026a], so a check that samples a few batch sizes at model load could pass a shape that fails at others.

Serving determinism and cost. With Linear layers on cuBLAS, 6 of 7 serving configurations produced identical tokens and top-k logprobs across all conditions (Table 3); controls with the mode off diverged on 4% to 76% of requests. The exception, Gemma-2-9B, fails under vLLM's own mode too (Section 8). Decode loss at batch 32 falls to 0.2% for Qwen2.5-7B and 1.0% for Gemma-2-9B (Table 4); 16 of 17 measured cells meet our preregistered 5% limit. At two-way tensor parallelism the residual loss (5.8%) comes from the mode's fixed-order communication, which costs 6.0% on its own. Routing only shapes that pass a load-time check, so that routers stay on the Triton kernel, keeps gpt-oss-20b's batch-invariance tests passing (4 of 4, the same as stock mode's 4), while routing every Linear layer fails all of them (0 of 4 pass), reproducing the failure that motivated the Triton default [vLLM contributors, 2026a].

Why we do not recommend it. cuBLAS is the cheapest route measured, but NVIDIA does not document its default kernels as batch-invariant, and the boundary we observe moves with K, with when the environment variables reach cuBLAS, and across batch sizes. The boundary is also GPU-specific: on a B200 with the same cuBLAS release, the failing shapes are exactly the odd widths from 97 to 127, the router passes, and 27 of 96 shapes have a different result than on H100.

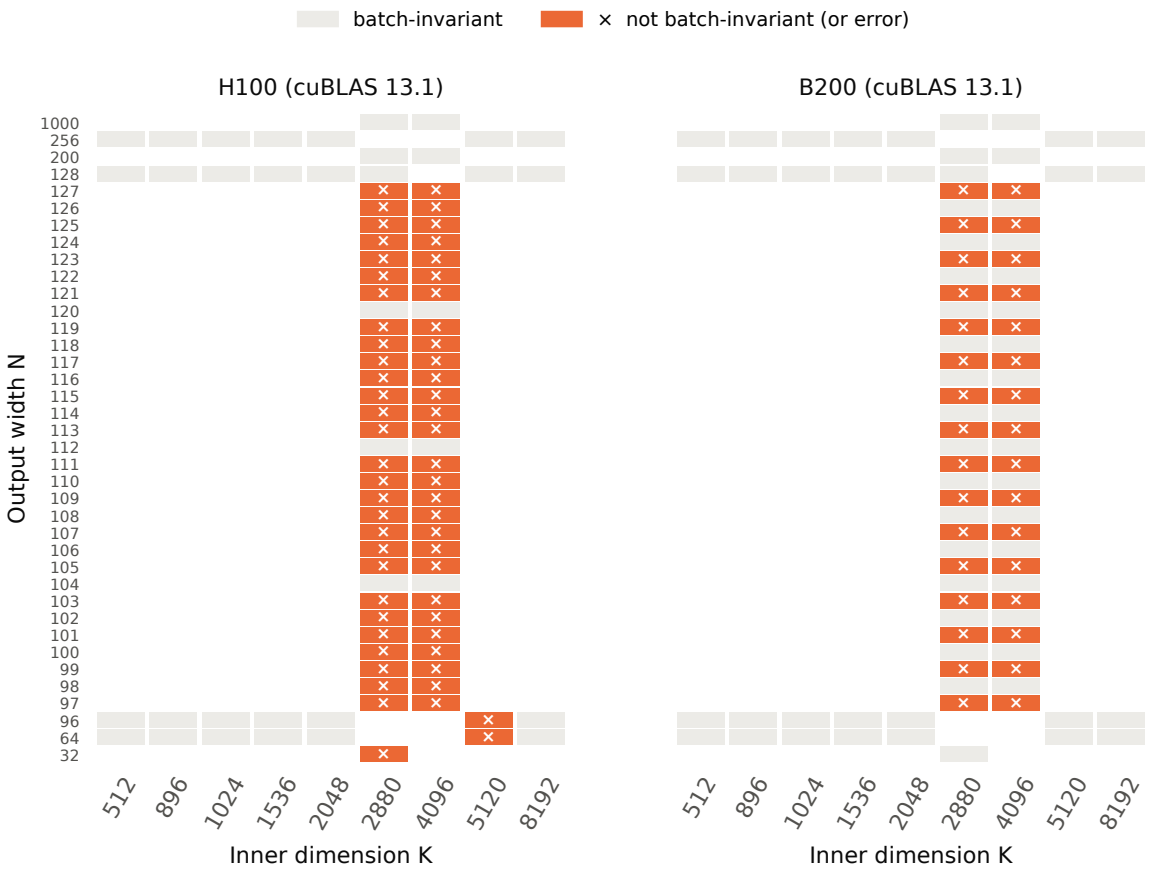


Figure 2: Which matrix shapes cuBLAS computes batch-invariantly under batch-invariant mode's settings (batch sizes 1 to 1024, then every sixteenth to 8192; bf16 and fp16), on H100 and B200 with the same cuBLAS release. Blank cells were not tested. On B200 the bf16 cells marked for odd widths raise an error rather than differ.

For bf16 odd widths the batch-invariant settings fail outright on B200: cuBLASLt rejects the shape and PyTorch's fallback refuses the split-K setting (5 shapes in the Blackwell sweep). An upstream change would have to rest on a property that a library update could silently remove.

7 Route 3: cuBLAS's batch-invariance mode on Blackwell

cuBLAS 13.4 documents an experimental batch-invariance mode for compute capability 10.x, enabled per call or with the environment variable `CUBLAS_BATCH_INVARIANCE_FLAGS`; it disables split-K and covers fp32, fp16, bf16, FP8 and FP4 [NVIDIA, 2026]. It is not in the cuBLAS release that vLLM pins. We loaded cuBLAS 13.8.2 in place of the pinned release and ran the bitwise test on a B200 with the mode set to automatic alignment, on 50 shape and dtype combinations: the router, the narrow shapes that fail on Hopper, odd widths, and large model shapes. Combined with the PyTorch settings that vLLM's batch-invariant mode applies (cuBLASLt preferred, reduced-precision reductions disabled), the mode does not pass our preregistered test: 65,200 of 1,460,100 comparisons differ, all on odd output widths (10 shape and dtype combinations), with 0 errors; without the mode, on the same library, 6,787 differ across 7. With PyTorch's default

Model	Stock	Plain cuBLAS	Plain, worst repeat
Qwen2.5-7B	63.1%	0.2%	3.1%
Gemma-2-9B	59.2%	1.0%	1.3%
Mistral-7B	5.8%	3.3%	5.9%
Qwen2.5-7B, TP=2	61.0%	5.8%	6.8%
Qwen2.5-32B, TP=2	58.2%	−2.3%	−1.0%

Table 4: Decode throughput loss at batch 32 against batch-invariant mode off. Median of three jobs.

Route	Decode loss, batch 32	Invariance	Status
Stock mode, untuned shapes	63.1%	by construction	vLLM default
Stock mode, tuned table	10.0%	by construction	upstream PR [Thompson
cuBLAS routing	0.2%	observed, shape-dependent	not recommended
cuBLAS batch-invariance mode (SM100)	not measured	documented	needs vLLM flag change

Table 5: Routes to batch-invariant serving for Qwen2.5-7B on H100 (cuBLAS mode on B200).

settings, however, the mode is batch-invariant everywhere we looked: 0 of 1,460,100 comparisons differ, including the router and every odd width, and in fp32 0 of 730,050. The failure is therefore an interaction between the mode and vLLM's settings rather than a gap in the mode. We first suspected the automatic alignment setting, which the documentation says is invariant only if every call detects the same alignment class, but a diagnostic rules it out: forcing a fixed 2-byte class makes matters worse (47,163 of 408,828 comparisons differ, against 35,094 with automatic alignment), and a 16-byte class is rejected for odd widths (6 errors). What breaks the mode is the reduced-precision setting through which vLLM disables split-K in PyTorch: with only cuBLASLt preferred and that setting left at its default, 0 of 408,828 comparisons differ on every shape. The mode already disables split-K, so the setting is redundant when the mode is on.

8 A serving-level failure: Gemma-2

Gemma-2-9B is not batch-invariant under vLLM's batch-invariant mode on H100: 40 of 100 requests diverged with stock mode (39 with cuBLAS routing, 60 with the mode off). Varying one component at a time (30 requests each) localizes it: switching attention to the Triton backend removes it (0 diverging, control 19), as does eager execution (0, control 23), disabling the sliding window (0, control 17), or turning off CUDA graphs while keeping compilation (0, control 15); removing logit softcapping does not (15, control 24). We reported this upstream; another contributor traced it to FlashAttention 3 dropping the sliding window when the window covers the batch's longest sequence, which changes the key-value tile between captured CUDA graphs and eager steps, and fixed it by keeping the window active under batch invariance, a change that has since been merged [vLLM contributors, 2026c]. On our H100 the issue's reproducer goes from 5 of 20 prompts differing between alone and batched to 0 of 20 with the fix.

9 Recommendations

For practitioners who need reproducible outputs today (Table 5): use vLLM's batch-invariant mode, check whether the model's Linear shapes are in the tuned table for the GPU, and if not,

tune them; the tuning script and the resulting entries are in our repository. Do not route to cuBLAS on Hopper. On Blackwell, cuBLAS's batch-invariance mode is the promising route once vLLM ships a cuBLAS release that includes it, provided vLLM does not also apply its PyTorch reduced-precision setting when the mode is on; its throughput cost is still to be measured.

10 Limitations

We tested one engine, single-node tensor parallelism up to four GPUs, dense models up to 32B parameters, and bf16; we did not test multi-node serving, quantized models, or production traffic. The tuning result covers one model at tensor parallelism one, and its decode measurements are single jobs. cuBLAS behavior was measured on the library versions listed and may change. We compare only against vLLM's own batch-invariant mode, not against other engines' deterministic modes or LLM-42 [Gond et al., 2026].

11 Related Work

He and Thinking Machines Lab [2025] identify batch-size-dependent reductions as the main source of inference nondeterminism and describe batch-invariant kernels; vLLM's batch-invariant mode follows this design, as does SGLang's deterministic mode, which keeps chunked prefill, CUDA graphs and prefix caching [SGLang Team, 2025]. Yuan et al. [2025] show that numerical nondeterminism changes reasoning accuracy across batch sizes, GPUs, and precisions. LLM-42 [Gond et al., 2026] obtains determinism through verified speculation rather than invariant kernels. Tuned configurations for vLLM's batch-invariant kernel have been contributed for individual models [vLLM contributors, 2026b,d]; we measure how much of the cost they recover and where they stop.

12 Conclusion

Reproducible serving has been treated as a trade between determinism and throughput. For dense models on current NVIDIA GPUs, most of that trade is a kernel-tuning gap: vLLM's batch-invariant kernel, configured for the model's shapes, keeps outputs bitwise identical at a fraction of the default cost. The vendor library is faster still, but its invariance is undocumented on Hopper and shape-dependent; on Blackwell, its documented batch-invariance mode passes our bitwise test unless vLLM's reduced-precision setting is also applied, so adopting it requires dropping that one setting on SM100. Code, raw outputs, and the decision rules for every experiment are at `https://github.com/imthomp/vllm-batch-invariance`.

[USER: AI-assistance statement; venue and arXiv category (cs.CL primary, cross-list cs.LG and cs.DC)]

References

Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation, 2026.

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

NVIDIA. cuBLAS library documentation, section "experimental: Batch invariance". cuBLAS 13.4 documentation, 2026. URL https://docs.nvidia.com/cuda/cublas/index.html.

SGLang Team. Towards deterministic inference in SGLang and reproducible RL training. LMSYS Org blog, September 2025. URL https://lmsys.org/blog/2025-09-22-sglang-deterministic/.

Isaac M. Thompson. [performance]: Batch-invariant mode on Hopper/Blackwell routes linear layers to Triton although cuBLAS is already batch-invariant under the mode's settings. vLLM issue 65569, 2026a. URL https://github.com/vllm-project/vllm/issues/65569.

Isaac M. Thompson. [kernel][perf] add H100 BF16 tuned configs for Qwen2.5-7B to the batch-invariant matmul table. vLLM pull request 65931, 2026b. URL https://github.com/vllm-project/vllm/pull/65931.

vLLM contributors. [bugfix][model] fix gpt-oss batch invariance. vLLM pull request 35404, merged February 2026, 2026a. URL https://github.com/vllm-project/vllm/pull/35404.

vLLM contributors. [kernel][perf] add H100 tuned configs for Llama-3.1-8B and Qwen3-8B to the batch-invariant matmul table. vLLM pull request 62219, 2026b. URL https://github.com/vllm-project/vllm/pull/62219.

vLLM contributors. [bugfix][attention] keep FA sliding window active under batch invariance. vLLM pull request 65627, 2026c. URL https://github.com/vllm-project/vllm/pull/65627.

vLLM contributors. [kernel][perf] add Blackwell BF16 tuned configs for Qwen2.5-1.5B to the batch-invariant matmul table. vLLM pull request 65903, 2026d. URL https://github.com/vllm-project/vllm/pull/65903.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025.